

colab.research.google.com/drive/1617IFC_2jGazCS05yVim3kyfxcq-qnEO#scrollTo=y57x2vcpEUz

colab.research.google.com/drive/1617IFC_2jGazCS05yVim3kyfxcq-qnEO#scrollTo=y57x2vcpEUz

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

Share

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

Share

```
neighbor_node = PuzzleNode(neighbor_state, g_cost, h_cost, current_node)
heapq.heappush(open_list, neighbor_node)

return None

def print_solution(path):
    """Formats and prints the puzzle path along with g(n), h(n), and f(n)."""
    if not path:
        print("\n[] No solution found. The puzzle may be unsolvable from this configuration.")
        return

    print(f"\n-----")
    print(f"Optimal Solution Found in {len(path) - 1} Moves!")
    print(f"-----")
    for index, node in enumerate(path):
        print(f"--- Step {index} ---")
        for row in node.state:
            print(" ".join(str(tile) if tile != 0 else " " for tile in row))
        print(f"  g(n) = {node.g_cost} (moves from start)")
        print(f"  h(n) = {node.h_cost} (Manhattan distance to goal)")
        print(f"  f(n) = {node.f_cost} (total state cost)")
        print("-" * 25 + "\n")

def get_user_matrix(matrix_name):
    """Prompts the user to input a valid 3x3 grid matrix configuration."""
    print(f"\n--- Enter the configuration for the {matrix_name} ---")
    print("Use numbers 1-8 for tiles, and 0 or '.' for the blank space.")
    print("Separate numbers in a row by spaces (e.g., '1 2 3').")

    while True:
        try:
            grid = []
            for i in range(3):
                row_input = input(f"Enter row {i+1}: ").strip().replace(' ', '0')
                row_parsed = [int(x) for x in row_input.split()]
                if len(row_parsed) != 3:
                    raise ValueError("Each row must contain exactly 3 numbers.")
                grid.append(tuple(row_parsed))
            full_matrix = tuple(grid)


```

Variables Terminal

4:11 PM Python 3

colab.research.google.com/drive/1617IFC_2jGazCS05yVim3kyfxcq-qnEO#scrollTo=y57x2vcpEUz

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

grid.append(tuple(row_parsed))

full_matrix = tuple(grid)

flattened = [item for row in full_matrix for item in row]

if sorted(flattened) != list(range(9)):

print("[Error] Invalid configuration! The board must contain numbers from 0 to 8 with no duplicates.")

continue

return full_matrix

except ValueError as e:

print(f"[Error] Invalid input: {e}. Please re-enter the entire grid matrix.")

if __name__ == "__main__":

print("=====")

print(" 8-Puzzle Solver using A* Search Algorithm")

print("=====")

initial_state = get_user_matrix("INITIAL STATE")

goal_state = get_user_matrix("GOAL STATE")

print("\n[Thinking] Processing search graph path...")

solution_path = solve_8_puzzle(initial_state, goal_state)

print_solution(solution_path)

8-Puzzle Solver using A* Search Algorithm

--- Enter the configuration for the INITIAL STATE ---

Use numbers 1-8 for tiles, and 0 or '.' for the blank space.

Separate numbers in a row by spaces (e.g., '1 2 3').

Enter row 1: 2 8 3

Enter row 2: 1 6 4

Enter row 3: 0 7 5

--- Enter the configuration for the GOAL STATE ---

Use numbers 1-8 for tiles, and 0 or '.' for the blank space.

Separate numbers in a row by spaces (e.g., '1 2 3').

Enter row 1: 1 2 3

Enter row 2: 8 0 4

Enter row 3: 3 7 6

Variables

Terminal

4:11 PM Python 3

Inbox (1,180) - deeksharajcs2

Untitled2.ipynb - Colab

Untitled2.ipynb - Colab

colab.research.google.com/drive/1617IFC_2jGazCS05yVim3kyfxcq-qnEO#scrollTo=vj57x2vcpEUz

Share

RAM Disk

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

```
=====
8-Puzzle Solver using A* Search Algorithm
=====

--- Enter the configuration for the INITIAL STATE ---
Use numbers 1-8 for tiles, and 0 or '.' for the blank space.
Separate numbers in a row by spaces (e.g., '1 2 3').
Enter row 1: 2 8 3
Enter row 2: 1 6 4
Enter row 3: 0 7 5

--- Enter the configuration for the GOAL STATE ---
Use numbers 1-8 for tiles, and 0 or '.' for the blank space.
Separate numbers in a row by spaces (e.g., '1 2 3').
Enter row 1: 1 2 3
Enter row 2: 8 0 4
Enter row 3: 7 6 5

[Thinking] Processing search graph path...

=====
Optimal Solution Found in 6 Moves!
=====

--- Step 0 ---
2 8 3
1 6 4
_ 7 5
g(n) = 0 (moves from start)
h(n) = 6 (Manhattan distance to goal)
f(n) = 6 (total state cost)
=====

--- Step 1 ---
2 8 3
1 6 4
7 _ 5
g(n) = 1 (moves from start)
h(n) = 5 (Manhattan distance to goal)
f(n) = 6 (total state cost)
=====

--- Step 2 ---
2 8 3
1 _ 4
```

Variables Terminal

4:11 PM Python 3

colab.research.google.com/drive/1617IFC_2jGazCS05yVim3kyfxcq-qnEO#scrollTo=vj57x2vcpEUz

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

```
--- Step 2 ---
2 8 3
1 4
7 6 5
g(n) = 2 (moves from start)
h(n) = 4 (Manhattan distance to goal)
f(n) = 6 (total state cost)
-----

--- Step 3 ---
2 3
1 8 4
7 6 5
g(n) = 3 (moves from start)
h(n) = 3 (Manhattan distance to goal)
f(n) = 6 (total state cost)
-----

--- Step 4 ---
2 3
1 8 4
7 6 5
g(n) = 4 (moves from start)
h(n) = 2 (Manhattan distance to goal)
f(n) = 6 (total state cost)
-----

--- Step 5 ---
1 2 3
8 4
7 6 5
g(n) = 5 (moves from start)
h(n) = 1 (Manhattan distance to goal)
f(n) = 6 (total state cost)
-----

--- Step 6 ---
1 2 3
8 4
7 6 5
g(n) = 6 (moves from start)
h(n) = 0 (Manhattan distance to goal)
f(n) = 6 (total state cost)
-----
```

Variables Terminal

snipping tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Markup and share